



ELEC3609/9609 - Week 10

Lab

09 Deploy Your Project

In this lab, you will learn how to deploy your Django web application to a cloud-based production environment using AWS EC2, uWSGI, and nginx. This hands-on exercise walks through the complete deployment pipeline, from launching an EC2 instance to serving your app via a web server, while highlighting best practices and common pitfalls. It prepares you for real-world deployment scenarios and aligns with the expectations for your Assignment 3 submission.

Individual Task: N/A

Lab Resources: [AWS Academy Learner Lab - Student Guide](#) and [AWS Academy Learner Lab - README](#).

Learning Objectives

After completing this lab, students should be able to:

- Set up and connect to an AWS EC2 instance using SSH and a secure key pair.
- Install necessary software packages including Python, pip, Django, uWSGI, and nginx.
- Configure nginx and uWSGI to serve a Django application in a production setting.
- Modify Django settings for deployment, including ALLOWED_HOSTS and DEBUG mode.
- Understand the importance of secure access, file permissions, and service orchestration for deployment.

Project Deployment (Ubuntu EC2 + nginx + uWSGI + SQLite)

 INFO

Deploying the project is an assessment. Although the lab contains the core steps needed to deploy your project, extra steps will be required to satisfy all the requirements of Assignment 3. In addition, the guide does not cover every static-file and media-file configuration (for example, CSS, JavaScript, and uploaded images). You may need to add extra configuration to the nginx configuration file.

Step 0: Preparation

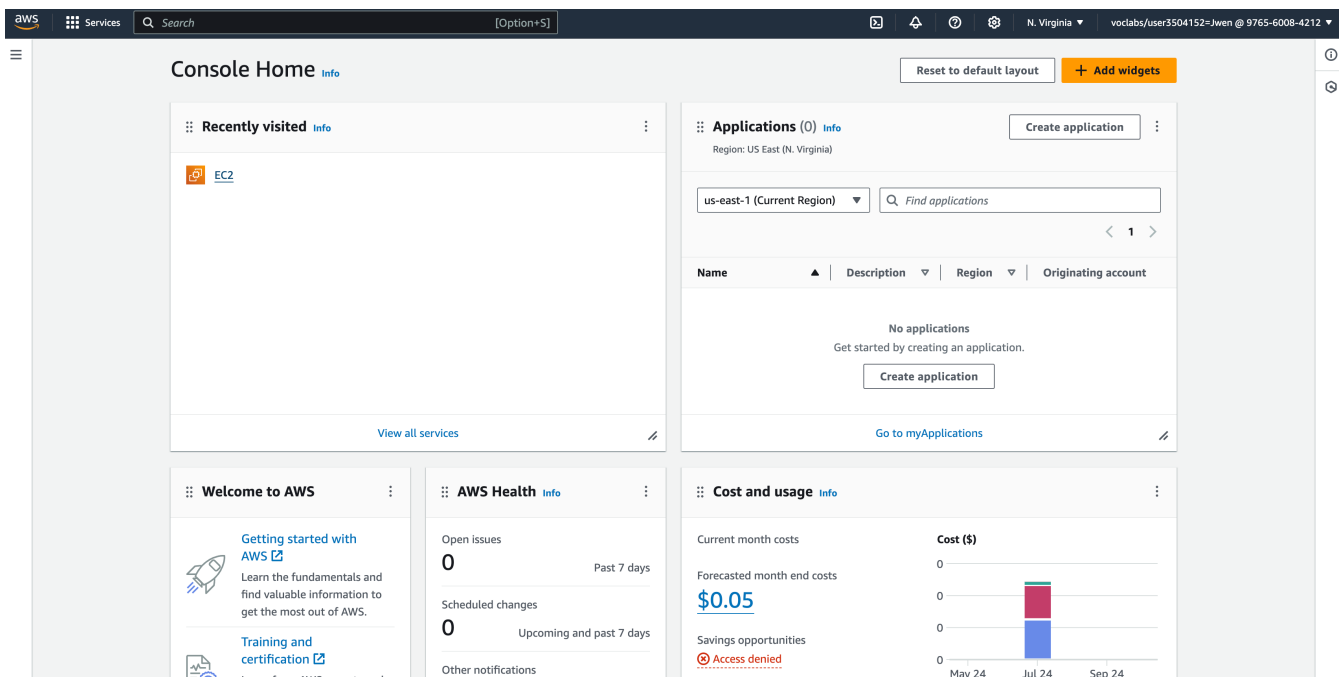
Check out the [AWS Academy Learner Lab - Student Guide](#) and [AWS Academy Learner Lab - README](#) before starting.

 TIP

Ensure you have a stable network connection and be patient while the AWS Academy Lab builds the environment, as it may take some time to complete the setup and ensure everything functions correctly.

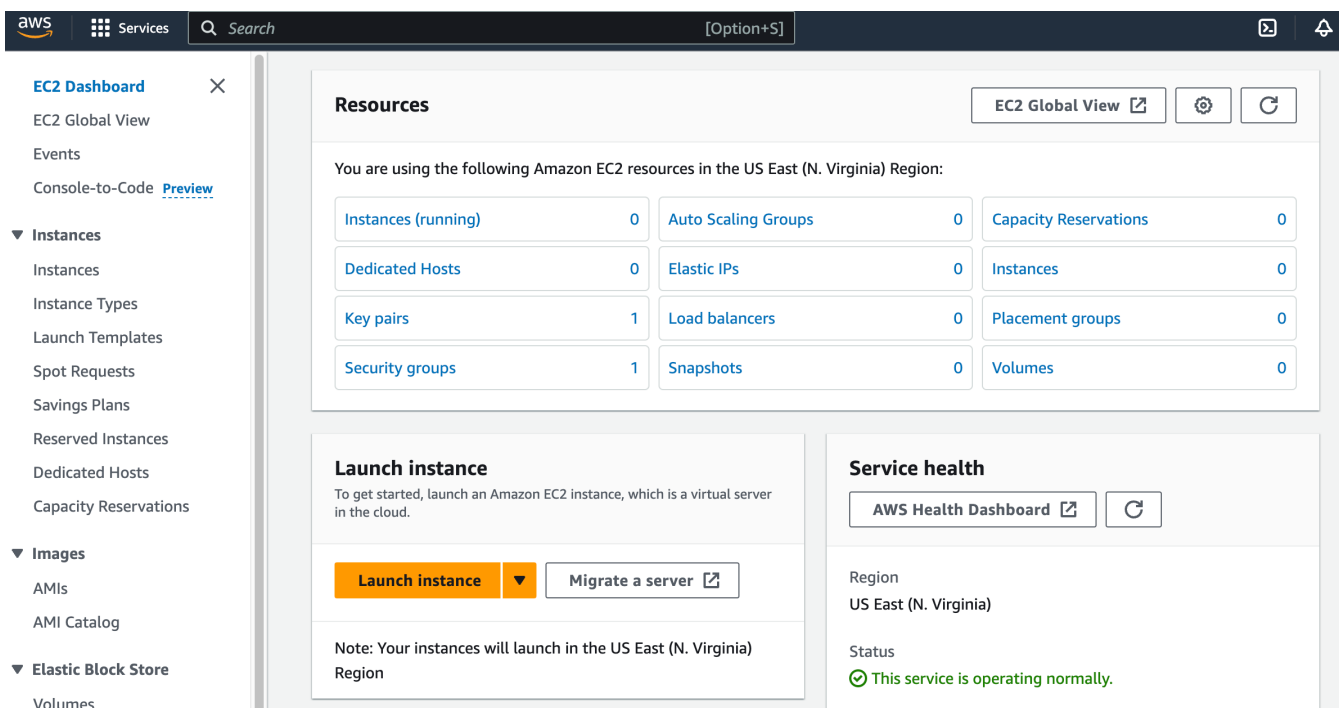
Step 1: Access EC2

In the AWS Console, navigate to the EC2 section by selecting it from the dashboard. If you do not see EC2 in the list, you can search for it using the search bar at the top of the console.



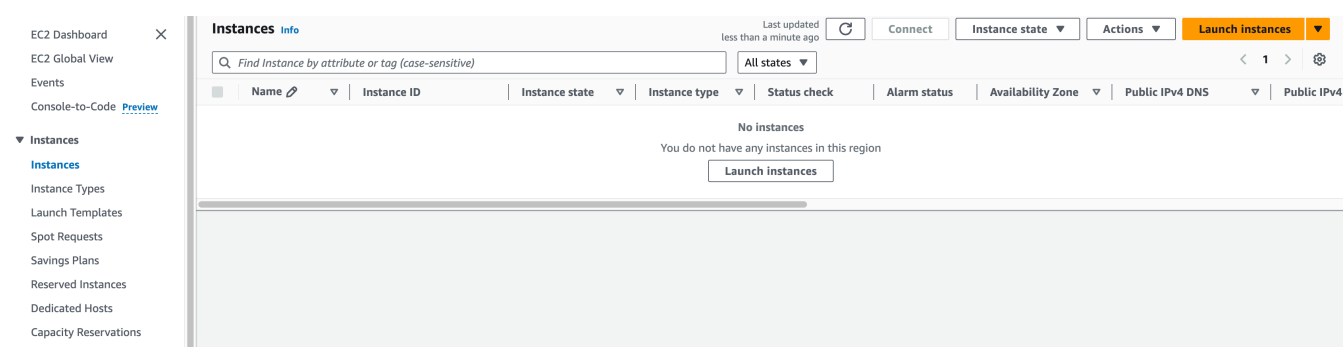
Step 2: View Running Instances

Click on **Instance (running)** to view any active instances you have running. You will need to launch a new one by following the next steps. This is where you can also monitor the status of any existing instances.



Step 3: Launch a New Instance

Click on **Launch instances** to create a new virtual server.



Step 4: Choose the Operating System

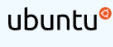
For the deployment, Ubuntu is recommended.

▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. Search or Browse for AMIs if you don't see what you are looking for below

 Search our full catalog including 1000s of application and OS images

Quick Start




[Browse more AMIs](#)
Including AMIs from
AWS, Marketplace and
the Community

Amazon Machine Image (AMI)

Ubuntu Server 22.04 LTS (HVM), SSD Volume Type

Free tier eligible ▼

ami-005fc0f236362e99f (64-bit (x86)) / ami-07ee04759daf109de (64-bit (Arm))

Virtualization: hvm ENA enabled: true Root device type: ebs

Description

Ubuntu Server 22.04 LTS (HVM),EBS General Purpose (SSD) Volume Type. Support available from Canonical (<http://www.ubuntu.com/cloud/services>).

Architecture

64-bit (x86) ▼

AMI ID

ami-
005fc0f236362
e99f

Username

ubuntu



Verified provider

Step 5: Select the Instance Type

Choose an instance type that fits your needs. Remember, you have \$50 in AWS credits, so it's important to select an instance that won't exceed your budget. The instances are charged at an hourly rate. Lower-tier instances like `t2.micro` or `t3.micro` are generally sufficient for your Django project deployment.

▼ Instance type [Info](#) | [Get advice](#)

Instance type

t2.micro

Free tier eligible

Family: t2 1 vCPU 1 GiB Memory Current generation: true
On-Demand Windows base pricing: 0.0162 USD per Hour
On-Demand SUSE base pricing: 0.0116 USD per Hour
On-Demand RHEL base pricing: 0.026 USD per Hour
On-Demand Linux base pricing: 0.0116 USD per Hour

☐ All generations

[Compare instance types](#)

[Additional costs apply for AMIs with pre-installed software](#)

Step 6: Create a Key Pair

Click **Create new key pair** to generate a `.pem` file, which is essential for accessing your EC2 instance securely via SSH. Download the key pair to your local machine and keep it secure, as it will be used for authentication later.

▼ Key pair (login) [Info](#)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - *required*

Select

▼

 [Create new key pair](#)

WARNING

You must use a key file to connect to your instance, there will be penalties if you simply allow all the traffic for SSH to your instance.

Step 7: Configure Key Pair Permissions

Choose a key pair name you want. Keep other settings the same as the image below. After you click on **Create key pair**, a `.pem` file will be downloaded to your computer.

Create key pair

Key pair name

Key pairs allow you to connect to your instance securely.

Enter key pair name

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type

☒ RSA
RSA encrypted private and public key pair

☐ ED25519
ED25519 encrypted private and public key pair

Private key file format

☒ .pem
For use with OpenSSH

☐ .ppk
For use with PuTTY

 When prompted, store the private key in a secure and accessible location on your computer. **You will need it later to connect to your instance.** [Learn more](#) 

Cancel

Create key pair



TIP

For Windows User

After downloading the `.pem` file, right-click on it, select `Properties > Security > Advanced`, and disable inheritance. Ensure that only you, the admin, and the system have access to the file.

For macOS User

You can restrict access to the key using the command `chmod 700 YourFile.pem`.

Step 8: Set Network Settings

Before launching the instance, configure the network settings to match the recommended settings in the guide. You should allow traffic to your instances on ports 22 (SSH), 80 (HTTP), and 443 (HTTPS). Double-check the security group rules to make sure SSH access is only allowed from trusted IPs.

▼ Network settings [Info](#)

Edit

Network [Info](#)

vpc-048e1119f629f0e78

Subnet [Info](#)

No preference (Default subnet in any availability zone)

Auto-assign public IP [Info](#)

Enable

Additional charges apply when outside of free tier allowance

Firewall (security groups) [Info](#)

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Create security group

☐ Select existing security group

We'll create a new security group called 'launch-wizard-1' with the following rules:

☒ Allow SSH traffic from

Helps you connect to your instance

Anywhere
0.0.0.0/0

☒ Allow HTTPS traffic from the internet

To set up an endpoint, for example when creating a web server

☒ Allow HTTP traffic from the internet

To set up an endpoint, for example when creating a web server

⚠ Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

×

Step 9: Launch the Instance

After configuring all basic settings, click `Launch instance`. This will start your EC2 instance with the selected operating system and settings. It may take a few minutes for the instance to fully initialise.

Step 10: Connect to Your Instance

Once your instance is up and running, it will show a "running" status in the **Instances** dashboard. Click on the newly created instance to view its full details, then choose **Connect**.

Instance summary for i-0128591706f95e773 (ELEC3609 TUT) [Info](#)

Updated 1 minute ago

[Refresh](#) [Connect](#) [Instance state ▼](#) [Actions ▼](#)

Instance ID i-0128591706f95e773 (ELEC3609 TUT)	Public IPv4 address 54.165.46.8 open address	Private IPv4 addresses 172.31.47.183
IPv6 address -	Instance state Running	Public IPv4 DNS ec2-54-165-46-8.compute-1.amazonaws.com open address
Hostname type IP name: ip-172-31-47-183.ec2.internal	Private IP DNS name (IPv4 only) ip-172-31-47-183.ec2.internal	Elastic IP addresses -
Answer private resource DNS name IPv4 (A)	Instance type t2.micro	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto-assigned IP address 54.165.46.8 [Public IP]	VPC ID vpc-048e1119f629f0e78	Auto Scaling Group name -
IAM Role -	Subnet ID subnet-0204aa31f09bdfb55	
IMDSv2 Required	Instance ARN arn:aws:ec2:us-east-1:976560084212:instance/i-0128591706f95e773	

Details | Status and alarms | Monitoring | Security | Networking | Storage | Tags

▼ **Instance details** [Info](#)

Platform Ubuntu (Inferred)	AMI ID ami-005fc0f236362e99f	Monitoring disabled
--------------------------------------	--	-------------------------------

In the **SSH client** tab, you'll see a command that you can use in your terminal to connect to the instance. Ensure the file path to your **.pem** file is correct when you paste the command.

[EC2](#) > [Instances](#) > [i-0128591706f95e773](#) > [Connect to instance](#)

Connect to instance [Info](#)

Connect to your instance i-0128591706f95e773 (ELEC3609 TUT) using any of these options

[EC2 Instance Connect](#)[Session Manager](#)[SSH client](#)[EC2 serial console](#)

Instance ID

 [i-0128591706f95e773](#) (ELEC3609 TUT)

1. Open an SSH client.
2. Locate your private key file. The key used to launch this instance is ELEC3609.pem
3. Run this command, if necessary, to ensure your key is not publicly viewable.
 `chmod 400 "ELEC3609.pem"`
4. Connect to your instance using its Public DNS:
 `ec2-54-165-46-8.compute-1.amazonaws.com`

Example:

 `ssh -i "ELEC3609.pem" ubuntu@ec2-54-165-46-8.compute-1.amazonaws.com`

 **Note:** In most cases, the guessed username is correct. However, read your AMI usage instructions to check if the AMI owner has changed the default AMI username.

E.g., `ssh -i "ELEC3609.pem" ubuntu@ec2-54-165-46-8.compute-1.amazonaws.com`

```
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1015-aws x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro

System information as of Sun Oct  6 09:33:33 UTC 2024

System load:  0.0                Processes:            102
Usage of /:   21.5% of 7.57GB     Users logged in:     0
Memory usage: 20%                IPv4 address for eth0: 172.31.47.183
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '24.04.1 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Sun Oct  6 09:28:48 2024 from 61.68.109.213
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
```

Step 11: Update and Install Necessary Software

After connecting to the instance, update the package list: `sudo apt-get update`

Verify Python version: `python3 --version`

Then, install pip: `sudo apt-get install python3-pip`

Install Django: `pip3 install django`

And nginx: `sudo apt-get install nginx`

After the installations, check nginx status with `sudo service nginx status`, you should see the following:

```
Oct 06 09:43:00 ip-172-31-47-183 systemd[1]: Starting A high performance web server and a reverse proxy server...
Oct 06 09:43:00 ip-172-31-47-183 systemd[1]: Started A high performance web server and a reverse proxy server.
```

```
sudo service nginx start.
```


This error occurs when you attempt to install a Python package globally on a system where Python is managed by the operating system (for example, Ubuntu or Debian). Installing packages globally is not recommended because it can interfere with system-managed dependencies and potentially break built-in tools.

To avoid this issue, create a virtual environment and install your packages inside it instead. This keeps your project dependencies isolated and prevents system conflicts.

```
python3 -m venv .venv
source .venv/bin/activate
pip install <package_name>
```

Step 12: Install uWSGI

To handle the application serving, install uWSGI by running: `pip3 install uwsgi`

You will also need the uWSGI Python plugin, which can be installed with: `sudo apt-get install uwsgi-plugin-python3`

The `uwsgi-plugin-python3` is a package that provides the Python 3 plugin for uWSGI, allowing uWSGI to run Python applications. You can check full lists of uWSGI plugins at usr/lib/uwsgi/plugins/.

Step 13: Clone Your Project

Clone your project repository from GitHub into the EC2 instance. Make sure to install any dependencies and libraries required for your project after cloning.

Step 14: Modify nginx Configuration

Navigate to `/etc/nginx/nginx.conf` and modify the file to point to the root and static file directories for your project. Replace the placeholder paths with the actual paths to your project's files. Once the configuration is updated, restart nginx with `sudo service nginx restart`.

```
# Official Russian Documentation: http://nginx.org/ru/docs/
user www-data;
worker_processes auto;
error_log /var/log/nginx/error.log;
pid /run/nginx.pid;

# Load dynamic modules. See /usr/share/doc/nginx/README.dynamic.
include /usr/share/nginx/modules/*.conf;
# include /etc/nginx/modules-enabled/*.conf;

events {
    worker_connections 1024;
}

http {
    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
        '$status $body_bytes_sent "$http_referer" '
        '"$http_user_agent" "$http_x_forwarded_for"';

    access_log /var/log/nginx/access.log main;

    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;
    keepalive_timeout 65;
    types_hash_max_size 2048;

    include /etc/nginx/mime.types;
    default_type application/octet-stream;
```

```
# Load modular configuration files from the /etc/nginx/conf.d directory.
# See http://nginx.org/en/docs/nginx_core_module.html#include for more
information.
include /etc/nginx/conf.d/*.conf;

server {
    listen 80 default_server;
    listen [::]:80 default_server;

    root /var/www/til_project/;

    # Load configuration files for the default server block.
    include /etc/nginx/default.d/*.conf;

    location / {
        include uwsgi_params;
        uwsgi_pass 127.0.0.1:8000;
    }

    location /static {
        alias /var/www/til_project/static/;
    }

    error_page 404 /404.html;
    location = /40x.html {}

    error_page 500 502 503 504 /50x.html;
    location = /50x.html {}
}
}
```

Step 15: Create a uWSGI Configuration File

Go to your project's folder, create a `uwsgi.ini` file and configure it with the following settings that are appropriate for your project. This file will tell uWSGI how to serve your application. Ensure the `chdir`, `wsgi-file`, `module`, and `socket` settings are accurate based on your project's structure.

```
[uwsgi]
chdir = /var/www/til_project
wsgi-file = /var/www/til_project/til_project/wsgi.py
module = til_project.wsgi:application
```

```
socket = 127.0.0.1:8000
master = true
disable-logging = true
processes = 2
threads = 4
```

Step 16: Update Django Project Settings

In the `settings.py` file of your Django project, add your domain (Public IPv4 DNS in AWS instance dashboard) to the `ALLOWED_HOSTS` list. Additionally, change the `DEBUG` setting to `False` to ensure the project runs in production mode.

```
# SECURITY WARNING: don't run with debug turned on in production!

DEBUG = False

ALLOWED_HOSTS = [
    '.localhost',
    '127.0.0.1',
    '::1',
    'ec2-54-165-46-8.compute-1.amazonaws.com'
]
```

Step 17: Start uWSGI

To start serving your Django project through uWSGI, run the command: `uwsgi uwsgi.ini --plugin python3`

This will initialise uWSGI with the configuration defined in your `uwsgi.ini` file.

Step 18: Access Your Project

You should be able to get access to your project through your URL now.

Notes

Please note that this lab only covers the basic steps to deploy a Django application. It does not include more advanced configurations required for Assignment 3, such as file permission management, proper static file serving, automatic service setup, or security controls. For more information, refer to the readings below.

DANGER

You should not run your Django project on the development server in your AWS instance, as this will result in a zero grade for project deployment. While it is possible to bypass some of the deployment steps by simply running the command:

```
python3 manage.py runserver 0.0.0.0:8000
```

This will start Django's development server, allowing you to access the application via your instance's IP address. However, this method is strictly for testing purposes and does not fulfil the requirements of Assignment 3.

Further Readings

Web & Application Server

- [Install and configure Nginx](#)
- [HTTP Load Balancing - Nginx](#)
- [Serving Static Content - Nginx](#)
- [Quickstart for Python/WSGI applications](#)

Django Deployment

- [Django Deployment checklist](#)
- [How to deploy static files - Django](#)
- [How to manage static files](#)

Linux Management

- [Linux file permissions explained - RedHat](#)

- [chmod](#)
- [chown](#)
- [systemd](#)

Terminal Tools

- [SSH](#)
- [Vim Cheatsheet](#)
- [Tmux Cheat Sheet](#)

Credit

This lab content was authored and maintained by **Jiawen Wen**, with materials adapted from prior offerings and updated to align with 2026 delivery. All standards used in this lab (IEEE, ISO/IEC) are referenced for educational purposes under fair use and are available to enrolled students via the university's licensed repository.